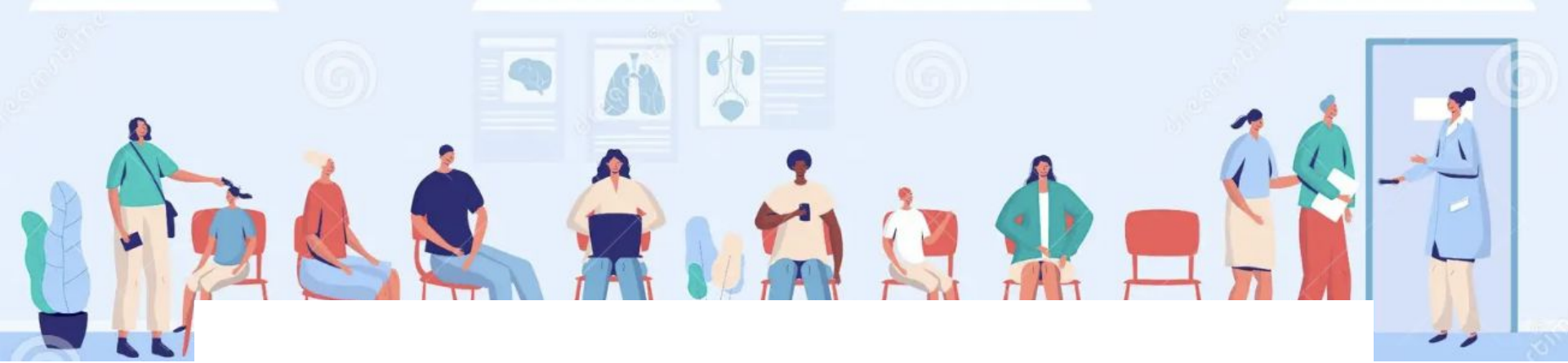


Efficient Patient Queue Management

Streamlined Scheduling Algorithms

The Frustration of Long Wait Times

Unveil the Problem with Impactful Statistics

60% of Patients Wait Too Long:

- Did you know a staggering 60% of patients report experiencing wait times exceeding 30 minutes in doctor's offices? (Source: National Patient Advocate Foundation)

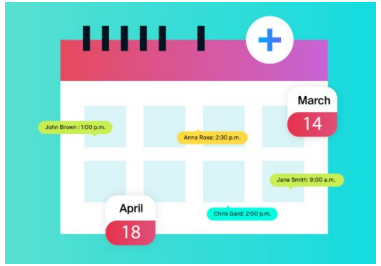
Beyond Frustration:

- These extended wait times don't just create frustration – they lead to dissatisfaction with the entire healthcare experience. Upset patients might even miss their visits altogether.

Impact on Everyone:

- The ripple effects of long waits extend beyond patients. Doctors also experience reduced productivity due to scheduling inefficiencies. This, in turn, can lead to unnecessary stress for healthcare providers.

Introduction



Why Efficient Scheduling Matters:

- Efficient scheduling isn't just about saving time—it's about ensuring everyone gets the care they need when they need it. By reducing waiting times and streamlining operations, we can enhance patient satisfaction and access to healthcare.

Objectives:

- Our main objective of this project is to Improve Patient Experience,Enhance Doctors Productivity,Reduce Administrative Burden and Ensure Resource Utilization.

Prioritizing Your Patients: A Smarter Scheduling Approach

Introducing the Solution: A System Built for Efficiency

Prioritize Based on Urgency:

- Move beyond the "first-come, first-served" approach. Implement a priority-based scheduling system that categorizes visits based on medical urgency. This ensures critical cases are seen first.

Estimated Visit Durations:

- Assign estimated visit durations for different visit types (e.g., checkups: 30 minutes, consultations: 45 minutes, emergencies: as needed). This allows for more realistic scheduling and reduces wait times for everyone.

Power of the Queue:

- Utilize a priority queue to manage appointments. Patients are automatically placed in the queue based on their assigned priority, ensuring urgent cases are seen promptly.

Patient Input

When patients call to our clinic, they begin by providing essential details for their visit. This involves entering their name and selecting the purpose of their visit from a list of options tailored to their needs. For instance, a patient like "Rishitha" may choose "Normal" as the purpose of her visit, indicating a routine health checkup.

Enter patient Name: rishitha
Pick visit purpose:
1)Normal-(Health Checkup)
2)Emergency(Operations - Accidents,Fractures)
3)Issues(Fever,Flu,cough,headache,diarrhea)

Queue Formation

- Once patients input their details, our system evaluates their visit purpose and allocates an estimated visiting time with the doctor while also determining the priority level for their concern.

Code Snippet:

```
if purpose == '1' :
        approxtime = 30
        priority = 0
    elif purpose == '2':
        approxtime = 90
        priority = 2
    elif purpose == '3':
        approxtime = 45
        priority = 1
    else :
        print('Wrong Input')
```

Stored in list format:

[‘Mehak’, 30, 0]
[‘Shruthi’, 45, 1]
[‘Tanya’, 30, 0]
[‘Samhitha’, 90, 2]
['Rishitha', 45, 1]
['Nakul', 90, 2]
[‘Hari’, 45, 1]
['Priya', 90, 2]

Queue Formation

Patients are added to the queue if their visit can be accommodated within the clinic's schedule. However, if the schedule reaches its maximum capacity for the day, the system notifies patients that no further appointments are available, and they should consider rescheduling for the following day. This ensures that patients are informed promptly and can plan their visits accordingly.

Code Snippet:

```
if time < 480 and (time + approxtime) < 480 :

doctorsqueue.append([name,approxtime,prio
rity])
        time = time+approxtime
    else :
        print (" \nReached max no of patients
today, please try tomorrow", name)
        break
```

Output:

Reached max no of patients today,please try tomorrow, asha

Order of Attendance - Priority Scheduling

Our system employs priority-based scheduling, categorizing the highest-priority cases to be attended by the doctor at the start of the day. If multiple patients share the same priority, we consider their approximate appointment times to prioritize the shortest appointments first. Given that we allocate the same time for patients with the same priority level, we subsequently organize the list based on a first-come-first-serve basis.

Code Snippet:

```
def priorityScheduling(proc, n):

    approx_time = []
    final_list = []
    # Sort processes by priority
    proc = sorted(proc, key=lambda proc:
(proc[2],proc[1]),reverse=True )
    quantum = 60

    print(" \n \n Order in which patients gets attended by doctor")
    for i in proc:
        print( i[0] + ',', end=" ")
        final_list.append(i[0])
        approx_time.append(i[1])

    findavgTime(proc, n,approx_time,quantum)
```

Output:

Order in which patients gets attended by doctor:
['Samhitha','Nakul','Priya,'Shruthi','Rishitha',
'Hari','Mehak','Tanya']

Wait, what about patients with scheduled or emergency operations?

Order of Attendance - Round Robin Scheduling

- Our system meticulously calculates the sequence in which patients will be attended by our medical staff, employing the Round Robin scheduling method to optimize efficiency.
- Each appointment is allocated a fixed time slice, known as the "quantum," ensuring equitable attention to all patients.
- This approach ensures fair allocation of resources by systematically rotating through the queue of patients, providing equitable attention to all.
- Round Robin is used to cyclically iterate through the queue of patients, allocating a fixed time slice for each appointment.
- Every patient gets a time quantum of 60 mins max with the doctor, as per the approx times the doctor can attend next patient if it is completed before 60 mins quantum.
- Operation patients receive an total 90 minutes for their appointments. First 60 mins for operation and then at the end of the day, an extra 30 minutes are allocated for checkups.
- Example: Patient A, with an Emergency visit for a fractured arm, would be attended to promptly, with the necessary time allocation to address their urgent medical condition.

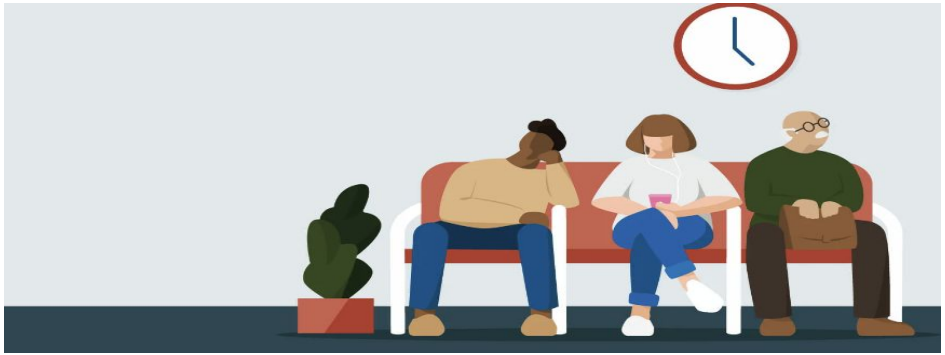
Average Waiting and Turnaround Times:

Our system meticulously calculates two critical metrics: average waiting time and turnaround time.Average waiting time reflects the duration patients spend waiting for their appointment to begin, whereas turnaround time encompasses the entire duration from check-in to check-out.

```
def findWaitingTime(processes, n, bt, wt, quantum, order):
    rem_bt = [0] * n
    for i in range(n):
        rem_bt[i] = bt[i]
    t = 0
    while(1):
        done = True
        for i in range(n):
            if (rem_bt[i] > 0) :
                done = False
                if (rem_bt[i] > quantum) :
                    t += quantum
                    rem_bt[i] -= quantum
                    order.append([processes[i], quantum])
                else:
                    order.append([processes[i], rem_bt[i]])
                    t = t + rem_bt[i]
                    wt[i] = t - bt[i]
                    rem_bt[i] = 0
        if (done == True):
            break
```

```
def findTurnAroundTime(processes, n, bt, wt, tat):
    for i in range(n):
        tat[i] = bt[i] + wt[i]
```

Average waiting time = 286.87500
Average turnaround time = 343.12500



Appointment Schedule:

- Our system dynamically generates a comprehensive schedule of appointments for each day based on the calculated order of attendance.
- The schedule includes essential details such as the patient's name, approximate visit time, and duration of the appointment.
- With visual representation, our appointment schedule offers clarity and organization. For instance, it may display that "Samhitha" has an appointment scheduled from 8:00 AM to 9:00 AM for an Emergency visit.

Code Snippet:

```
def schedule_appointments(start_time, durations):
    start_datetime = datetime.strptime(start_time,
"%I:%M %p")
    current_time = start_datetime
    schedule = []
    for duration in durations:
        end_time = current_time +
timedelta(minutes=duration)
        schedule.append((current_time.strftime("%I:%M
%p"), end_time.strftime("%I:%M %p")))
        current_time = end_time
    return schedule
```

Output:

Schedule:
Patient 1: nakul - 08:00 AM to 09:00 AM
Patient 2: ritwik - 09:00 AM to 10:00 AM
Patient 3: priya - 10:00 AM to 11:00 AM
Patient 4: ashi - 11:00 AM to 11:45 AM
Patient 5: satya - 11:45 AM to 12:30 PM
Patient 6: rishi - 12:30 PM to 01:00 PM
Patient 7: sana - 01:00 PM to 01:30 PM
Patient 8: sonu - 01:30 PM to 02:00 PM
Patient 9: nakul - 02:00 PM to 02:30 PM
Patient 10: ritwik - 02:30 PM to 03:00 PM
Patient 11: priya - 03:00 PM to 03:30 PM

THANK YOU
iconscout
Marina Zlochin